

Manual for Free Energy Surface Calculation for Bucket Sampling (Python Version 1.0, 13 June 2026)

Code: `bucket_reweight.py`

A single, self-contained Python program that reconstructs free-energy surfaces from **bucket (window) sampling with well-tempered metadynamics (WT-MetaD)**.

It reads **one** control file (`bucketsampling.inp`), runs the complete per-bucket reweighting pipeline inside each bucket folder, and stitches the buckets together into a 1D (and optionally 2D) free-energy surface. The program works for any dimensions, and for a selected set of CVs along which WT-MetaD is applied. Note that, per the original bucket-sampling protocol, WT-MetaD is expected along the bucket-CV.

See Ref [1] <https://doi.org/10.1021/acs.jctc.4c00776> for details.

Note that this code is experimental and differs from the code used in the above publication to make the entire postprocessing pipeline easier to implement with a single code base. The code has only been tested for a few examples. Thus, you may use the code with caution. If you find any bugs, report them to nnair@iitk.ac.in (Nisanth N. Nair, IIT Kanpur).

1. What it does

For each bucket folder it produces:

File	Meaning
<code>ct.dat</code>	WT-MetaD time-dependent constant $c(t)$ (one row per hill)
<code>vbias.dat</code>	Metadynamics bias $V(t)$ (one row per MD/COLVAR frame)
<code>Pu.dat</code>	Reweighted 1D probability and $-kT \ln P$ along the bucket CV
<code>gradients.dat</code>	Mean force dF/ds on the bucket's inner grid (spline)

Then, in the directory where you run it, it writes the stitched results:

File	Meaning
<code>fes_along_bucket.dat</code>	1D free energy $F(s)$ along the bucket CV (through mean-force intergration method)
<code>free_energy_2D.dat</code>	2D free energy surface $F(s_1, s_2)$ (only when a PRINT2D block exists)

2. Requirements

- **Python 3.8+**
- **NumPy** and **SciPy** (required)
- **mpi4py** + an MPI runtime (OpenMPI / MPICH providing `mpirun`) - *optional*, only needed to run the `vbias` step across multiple cores/nodes with true MPI.
- **numba** - *optional*, enables a real-OpenMP (prange) kernel for `vbias` (`--backend numba`). Without it the default NumPy backend is used.

Quick environment check:

```
python -c "import numpy, scipy; print('numpy/scipy OK')"
```

For MPI:

```
command -v mpirun && python -c "import mpi4py; print('mpi4py OK')"
```

Optional:

```
python -c "import numba; print(numba.__version__)"
```

If a package is missing, install it into your environment, e.g.:

```
pip install numpy scipy mpi4py numba
```

3. Directory layout

Run the program from the directory that contains `bucketsampling.inp`. Each bucket folder named in that file must already contain a COLVAR and a HILLS file from the simulation:

```
my_run/
├── bucketsampling.inp
├── w1/
│   ├── COLVAR
│   └── HILLS
├── w2/
│   ├── COLVAR
│   └── HILLS
└── ...
```

Use `--root DIR` if the bucket folders live somewhere other than the current directory.

4. Input file: `bucketsampling.inp`

Free-format, one record per line; trailing `# ...` comments and blank lines are ignored. Fortran-style reals (`1.0d0`, `-3.2D0`) are accepted. The input has 3 blocks:

4.1 First Block (Entries are to be in this exact order)

Line No.	Input Parameter	Description	Data Type
1	nbucket	Number of buckets	Integer type
2	ncv	Total number of CVs considered. Expecting the same number for all buckets	Integer type.
3	iwrap(1 to ncv)	Indicate by 0 or 1, if each CV is to be wrapped between $-\pi$ to $+\pi$. "0" means, not to wrap, and "1" means wrap. This is the case for TORSION type CVs.	ncv number of integers separated by space. Only accepted values are 0 or 1. E.g. in the case of phi and psi torsion angles (as the only two CVs considered), then this line should be 1 1
4	icv_metad(1 to ncv)	Indicate by 0 or 1, for each CV, whether WT-MetaD bias is acting on that CV or not. "0" means NO and "1" means YES.	ncv number of integers separated by space. Only accepted values are 0 or 1. E.g. in the case of two CVs, where only one the CV1 is having WT-MetaD bias, while it is not the case for CV2, then this line should be 1 0 The original bucket sampling work is expecting one to use WT-MetaD bias along the bucket CV. This code is also expecting that at least one CV is having WT-MetaD bias.
5	icv_bucket	Index of the CV along which the bucket bias is applied.	One integer number.
6	T_phys T_ext	Temperature of the physical degrees of freedom and the extended degrees of freedom in K. If extended degrees of freedom is not used, then keep the T_ext same as T_phys.	Two real numbers separated by space is read. Note that the T_phys is not used in the code. If one is using the bucket sampling without the extended Lagrangian formalism, then use T_phys and T_ext the same.
7	bias_fact	Bias factor for WT-MetaD: $(T + \Delta T)/T$	A real number is read.

4.2 Second Block: It has nbucket sub-blocks

For each bucket, the following sub-block should be present in increasing order of the bucket window. Bucket windows should be arranged in a manner that the η_L values (i.e. the starting grid value) of the buckets are in increasing order.

folder	Folder name containing the results of the respective bucket, especially the COLVAR and HILLS files. See Directory Layout (Section 3) for details.	} For ncv CVs
grid_min grid_max d_grid	For CV1 (3 real numbers)	
grid_min grid_max d_grid	For CV2 (3 real numbers)	
...	..	
t_min t_max	Two integers are read. They are the MD time step range (min and max) to be considered for reconstructing the probability distribution (and the final free energy surface). That is, from t_min to t_max steps in COLVAR file are only used for reconstruction of probability distribution. If t_max is a negative number, then t_max is set equal to the maximum time steps present in the COLVAR file.	
cv_print_stride hill_update_stride	Two integers are read. These are the printing frequency (or stride) of CVs in the COLVAR file and the frequency (stride) for updating the bias potential (i.e. HILLS deposition stride) in the WT-MetaD simulation.	

Note that grid_min and grid_max for the bucket CV are expected to be η_L the η_U values of the bucket CV (See Ref.).

The above sub-block needs to be repeated for nbucket number of buckets.

4.2 Third Block: Optional

This is an optional block. The block starts with the keyword “PRINT2D”. The structure of the third block is below.

PRINT2D	A text keyword that triggers the code to compute $F(s_1, s_2)$. Without this keyword, the code will exit by computing only the $F(s_1)$. Here s_1 is the CV along the bucket-bias, and s_2 is any other CV.
icv2 grid_min grid_max dgrid	Format: Integer <space> real <space> real <space> real. Here, icv2 is the index of the CV along which the 2D free energy surface needs to be computed.

5. Running it

5.1 With the launcher script (recommended)

```
cd my_run # directory containing bucketsampling.inp
./run_bucket_reweight.sh [NRANKS] [THREADS] [BACKEND] [INPUT] [ROOT] [FORCE]
```

Arg	Meaning	Default
NRANKS	MPI ranks for the vbias step	2
THREADS	threads per rank for vbias	1
BACKEND	numpy or numba	numpy
INPUT	control file	bucketsampling.inp
ROOT	base dir holding the bucket folders	.
FORCE	force/1/yes → recompute ct.dat/vbias.dat	(reuse existing)

The script sets the thread/MPI environment, then runs the driver once. Total cores used by vbias = **NRANKS** × **THREADS**; keep it ≤ your physical cores.

Examples:

```
./run_bucket_reweight.sh 4 1 numpy # 4 MPI ranks
./run_bucket_reweight.sh 8 1 numpy # 8 MPI ranks
./run_bucket_reweight.sh 4 2 numba # 4 ranks x 2 threads (numba)
./run_bucket_reweight.sh 1 8 numba # no MPI, 8 threads, 1 node
./run_bucket_reweight.sh 4 1 numpy bucketsampling.inp . force #recompute
ct.dat & vbias.dat
```

Override the Python interpreter with `PYTHON=...`, and the MPI temp dir with `MPI_TMPDIR=...` (defaults to /tmp, which avoids macOS PMIx tmpdir issues).

It is advisable to test the performance on your resources.

5.2 Calling the driver directly

```
python bucket_reweight.py [INPUT] [options]
```

Option	Meaning
INPUT	control file (default bucketsampling.inp)
--root DIR	base directory of the bucket folders (default .)
--nranks N	MPI ranks for the vbias step (default 1)
--threads T	threads per rank for vbias (default 1)
--backend B	numpy (default) or numba
--force	recompute ct.dat/vbias.dat even if they already exist
--no-reweight	stop after per-bucket files; skip the final stitching
-v, --verbose	extra per-grid / per-bucket diagnostics in the stitching step

Example:

```
export OMP_NUM_THREADS=1 TMPDIR=/tmp
```

```
python bucket_reweight.py bucketsampling.inp --root . --nranks 4 --backend  
numpy --force
```

6. How the parallelism works

The driver is **serial** — it processes one bucket folder at a time. Only the expensive vbiasstep is parallel. Because a serial process cannot turn itself into MPI ranks, the driver **re-launches itself under mpirun** in a hidden worker mode, once per bucket:

```
mpirun -np N python bucket_reweight.py <inp> --vbias-worker --bucket-index  
i ...
```

- You *never* type mpirun yourself, and you *never* run the driver under mpirun.
- Each rank computes its share of MD steps (round-robin for load balance); rank 0 writes vbias.dat.
- If mpirun/mpi4py are unavailable, or --nranks 1, vbias is computed **in-process**(NumPy, optionally numba threads) automatically.

Two knobs:

- **--nranks** → MPI processes (scale across cores/nodes).
- **--threads** → shared-memory threads per rank. Real OpenMP threads with --backend numba; faster vectorized/BLAS math with --backend numpy.

You'll see, per bucket, either

```
vbias.dat: COMPUTING (mpirun -np 4 x 1 threads, self-worker) ... # true  
MPI
```

vbias.dat: COMPUTING (single process x 1 threads) ...
process

in-

7. Restarting vs. recomputing

By default, if a bucket folder already contains `ct.dat` or `vbias.dat`, they are **restarted** (you'll see FOUND existing file -> REUSED). `Pu.dat` and `gradients.dat` are always rebuilt from them. To recompute the expensive files, pass `--force` (or the `FORCE` arg to the launcher); you'll then see COMPUTED.

8. Output file formats

ct.dat	hill_index c(t) (one row per HILLS line); See Eq. 12 of Ref. [1].
vbias.dat	md_step V_bias (one row per COLVAR frame; V is shifted so $V(\text{step } 1) = 0$); See Eq. 6 of Ref. [1].
Pu.dat	s_1 $P_1(s_1)$ $-kT \cdot \ln P_1(s_1)$ on the bucket CV grid (See Eq.10 of Ref. [1])
gradients.dat	s_1 $F_1(s)$ dF_1/ds_1 on the inner grid (See Eq. 9 of Ref.) nbin-1 points: gmin ... gmax-dgrid.
fes_along_bucket.dat	s_1 $F_1(s)$ (Stitched 1D profile) (See Eq. 9 of Ref.)
free_energy_2D.dat	s_1 s_2 $F(s_1, s_2)$ $P(s_1, s_2)$ with a blank line after each s_1 block (to make it gnuplot pm3d/splot-friendly); See Eq. 15, 8 of Ref. [1].

The filenames in **red** correspond to each bucket and will be written to the bucket's folder. The ones in **green** are after stitching the distributions of bucket windows. The path to this file is the running directory.

Note that energies are in **kJ/mol** ($k_B = 8.314472E-3$ kJ/mol/K); verify your simulation outputs use the same units. Therefore $c(t)$, $vbias$, $F(s_1)$, $F(s_1, s_2)$ are in kJ/mol.

Quick plots (gnuplot):

```
plot 'fes_along_bucket.dat' u 1:2 w lp                    # 1D
```

```
set pm3d map; set contours both; set cntrparam levels 20; splot  
'free_energy_2D.dat' u 1:2:3 # 2D
```

9. Typical workflow

```
cd my_run
```

```
# 1. First run: compute everything with 4 MPI ranks
```

```
./run_bucket_reweight.sh 4 1 numpy
```

```
# 2. Re-tune the 2D grid / window in bucketsampling.inp, then re-stitch fast
```

```
# (ct.dat & vbias.dat are reused automatically)
```

```
./run_bucket_reweight.sh 4 1 numpy
```

```
# 3. Changed temperature / bias factor / HILLS? force a clean recompute
```

```
./run_bucket_reweight.sh 4 1 numpy bucketsampling.inp . force
```

10. Troubleshooting

Message / symptom	Likely cause & fix
running vbias as a single threaded process (with <code>--nranks>1</code>)	mpi4py or mpirun not found. Install MPI; ensure mpirun is on PATH.
missing COLVAR/HILLS in <code><folder></code>	The bucket folder lacks input files, or <code>--rootpoints</code> to the wrong place.
HILLS columns (...) < expected ...	icv_metad count vs. HILLS columns mismatch ($\text{HILLS} = 1 + 2 * \text{ncv_mtd} + 1$).
gradients.dat for bucket k has N rows, expected M	Bucket grid changed after gradients.dat was written — run with <code>--force</code> .
grid spacing / gridmin / gridmax / tiling errors	Bucket grids along the bucket CV must share dgrid and tile contiguously.
t_max (...) > md_steps (...)	t_max exceeds the number of COLVAR frames; lower it or use <code>-1</code> for “all”.
i_mtd computation error	w_cv/w_hill inconsistent with HILLS length; check the deposition stride.
PMIx / tmpdir error from mpirun(macOS)	Set <code>MPI_TMPDIR=/tmp</code> (the launcher already does this).

For more diagnostics during the stitching step, add `-v`.

11. Files

File	Role
bucket_reweight.py	Main self-contained driver (run this)
run_bucket_reweight.sh	Launcher: sets thread/MPI env and runs the driver
bucketsampling.inp	Control input file (you provide/edit this)

Reference:

[1] <https://doi.org/10.1021/acs.jctc.4c00776>